

FinalProj_551

Jake Gwinn

2025-11-29

Exploratory Data Analysis

Read in the Data

```
train <- read_csv("train.csv")

## Rows: 2000 Columns: 21
## -- Column specification -----
## Delimiter: ","
## dbl (21): battery_power, blue, clock_speed, dual_sim, fc, four_g, int_memory...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
train_orig <- train

train %>% head()

## # A tibble: 6 x 21
##   battery_power blue clock_speed dual_sim    fc four_g int_memory m_dep
##   <dbl> <dbl>      <dbl>    <dbl> <dbl> <dbl>      <dbl> <dbl>
## 1         842     0         2.2        0     1     0         7  0.6
## 2        1021     1         0.5        1     0     1        53  0.7
## 3         563     1         0.5        1     2     1        41  0.9
## 4         615     1         2.5        0     0     0        10  0.8
## 5        1821     1         1.2        0    13     1        44  0.6
## 6        1859     0         0.5        1     3     0        22  0.7
## # i 13 more variables: mobile_wt <dbl>, n_cores <dbl>, pc <dbl>,
## #   px_height <dbl>, px_width <dbl>, ram <dbl>, sc_h <dbl>, sc_w <dbl>,
## #   talk_time <dbl>, three_g <dbl>, touch_screen <dbl>, wifi <dbl>,
## #   price_range <dbl>
```

Basic Data Cleaning

```
train <- train %>%
  mutate(
    blue = as.logical(blue),
    dual_sim = as.logical(dual_sim),
    four_g = as.logical(four_g),
    three_g = as.logical(three_g),
    touch_screen = as.logical(touch_screen),
    wifi = as.logical(wifi),

    n_cores = as.integer(n_cores),
```

```

    price_range = price_range + 1L
  ) %>%
  mutate(
    price_range = factor(price_range,
                        levels = c(1, 2, 3, 4),
                        ordered = TRUE)
  )

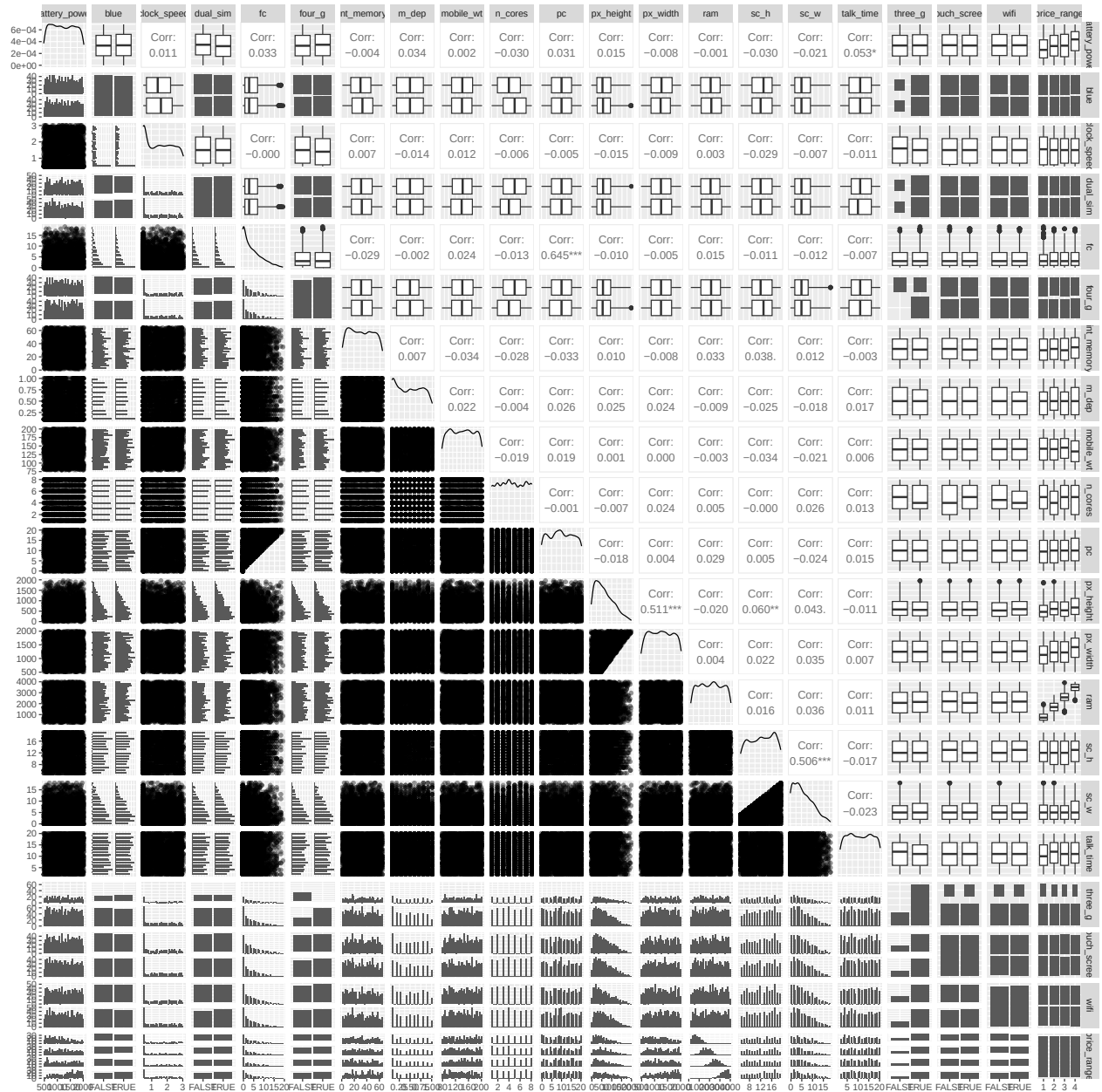
```

Plots

```

train %>%
  ggpairs(
    # aes(colour = price_range),
    diag = list(continuous = wrap("densityDiag", alpha = 0.5)),
    lower = list(continuous = wrap("points", alpha = 0.4))) +
  # ggpairs(
  #   diag = list(continuous = wrap("densityDiag", alpha = 0.5)), lower = list(continuous = wrap("
  scale_fill_brewer(palette = "Set2")

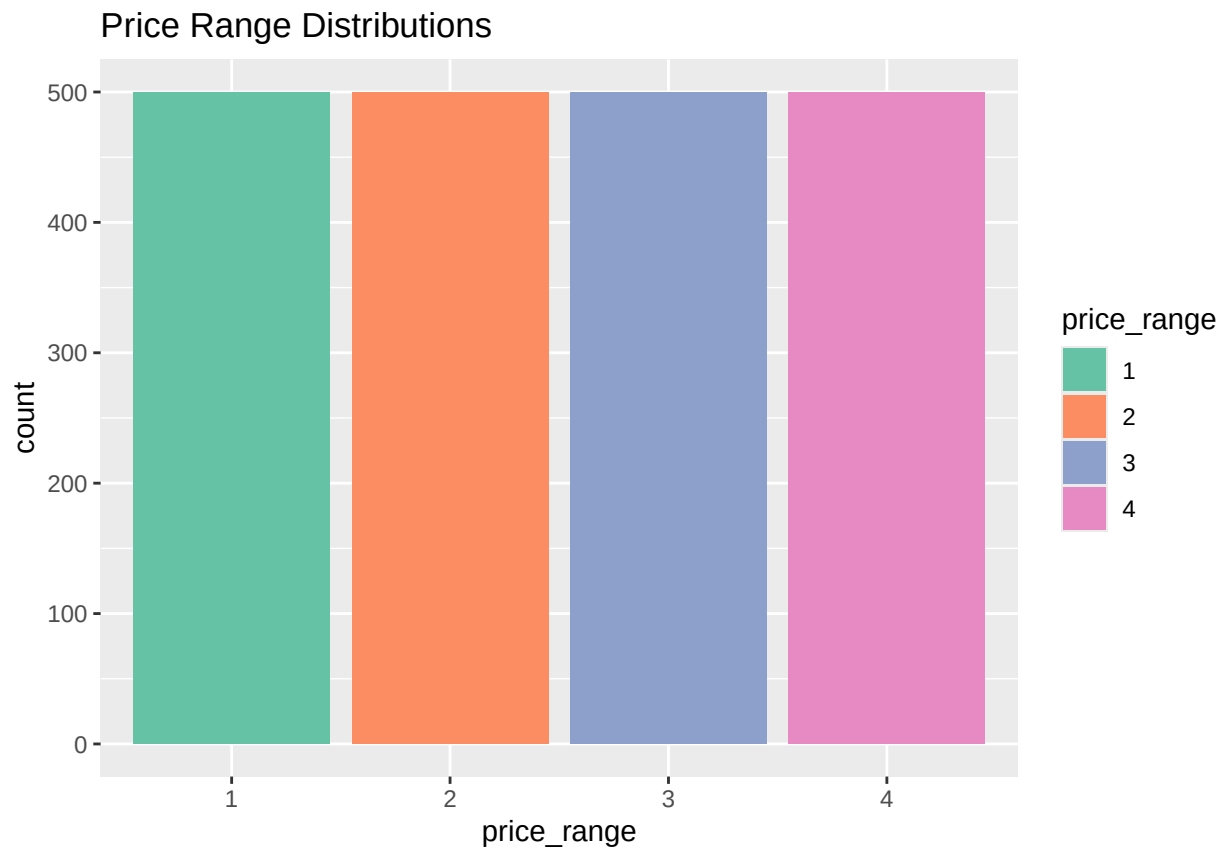
```



```
ggsave("pair_plot.png", dpi = 600)
```

- Pair plot for all variables

```
train %>%
  ggplot(aes(x=price_range, fill = price_range)) +
  geom_bar() +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title="Price Range Distributions"
  )
```

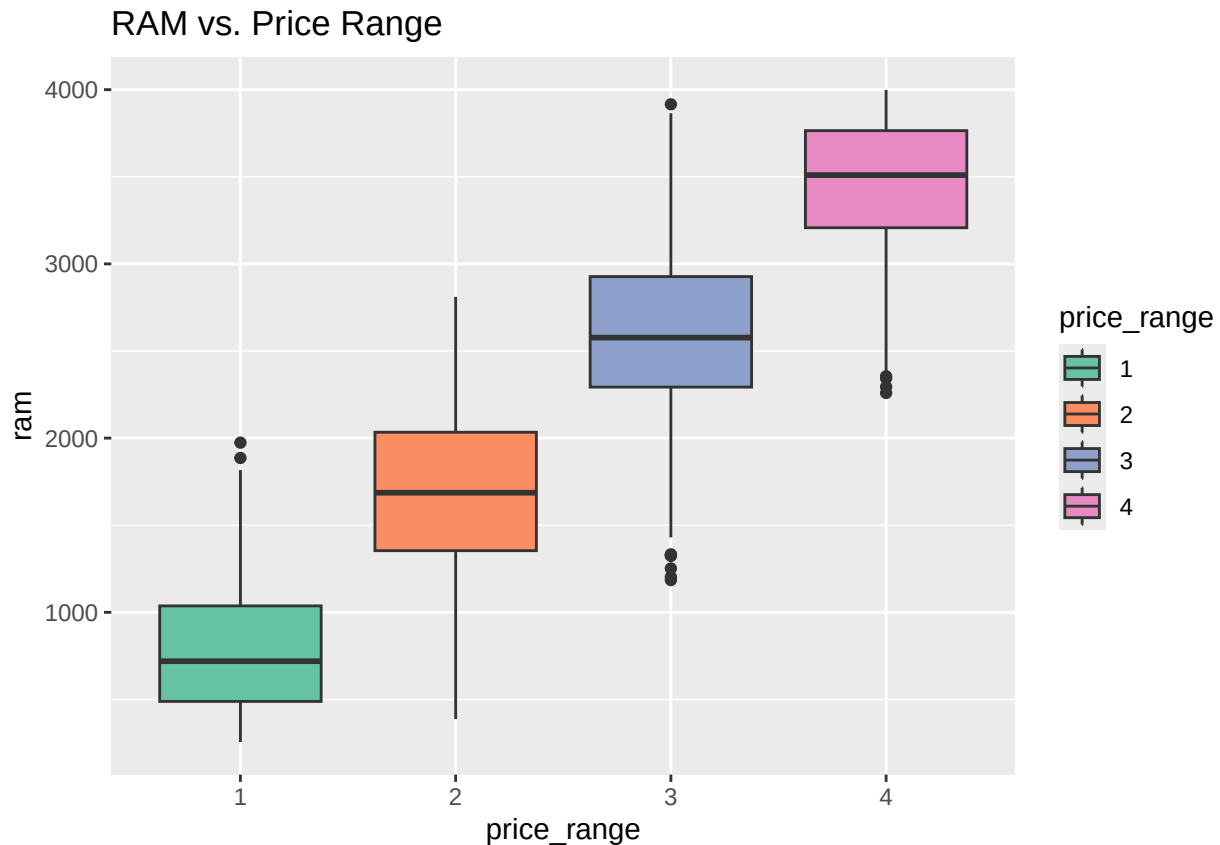


```
ggsave("price_plot.png", dpi = 600)
```

```
## Saving 6.5 x 4.5 in image
```

- Price category distributions

```
train %>%  
  ggplot(aes(x=price_range, y=ram, fill = price_range)) +  
  scale_fill_brewer(palette = "Set2") +  
  geom_boxplot() +  
  labs(  
    title = "RAM vs. Price Range"  
  )
```



```
ggsave("ram_vs_price_plot.png", dpi = 600)
```

```
## Saving 6.5 x 4.5 in image
```

- Ram vs price distribution showing monotonic relationship -> will be important

Modeling

Basic prep and Scaling

```
cat_cols <- c("blue", "dual_sim", "four_g", "three_g", "touch_screen", "wifi")
```

```
numeric_cols <- colnames(train)[!colnames(train) %in% cat_cols]
```

```
numeric_cols <- numeric_cols[numeric_cols != "price_range"]
```

```
df <- train_orig
```

```
df[numeric_cols] <- scale(train_orig[numeric_cols])
```

```
df <- df[c(numeric_cols, cat_cols)]
```

```
y <- train_orig$price_range + 1L
```

```
X <- as.matrix(df)
```

```
K <- length(unique(y))
```

```
N <- nrow(X)
```

```
D <- ncol(X)
```

```
data <- list(N=N, D=D, K=K, X=X, y=y)
```

```
code_stan <- "phone_fin_proj.stan"
writeLines(readLines(code_stan))
```

- This is the STAN code for the model

```
## Trying to compile a simple C file
```

6

```

## In file included from /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/library/StanHeader:
## In file included from /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/library/RcppEigen:
## In file included from /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/library/RcppEigen:
## /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/library/RcppEigen/include/Eigen/src/Core:
## 679 | #include <cmath>
##      |         ^~~~~~
## 1 error generated.
## make: *** [foo.o] Error 1
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.000162 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.62 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 1: Iteration:   500 / 5000 [ 10%] (Warmup)
## Chain 1: Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 1: Iteration:  1500 / 5000 [ 30%] (Warmup)
## Chain 1: Iteration:  2000 / 5000 [ 40%] (Warmup)
## Chain 1: Iteration:  2500 / 5000 [ 50%] (Warmup)
## Chain 1: Iteration:  2501 / 5000 [ 50%] (Sampling)
## Chain 1: Iteration:  3000 / 5000 [ 60%] (Sampling)
## Chain 1: Iteration:  3500 / 5000 [ 70%] (Sampling)
## Chain 1: Iteration:  4000 / 5000 [ 80%] (Sampling)
## Chain 1: Iteration:  4500 / 5000 [ 90%] (Sampling)
## Chain 1: Iteration:  5000 / 5000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 9.385 seconds (Warm-up)
## Chain 1:                9.909 seconds (Sampling)
## Chain 1:                19.294 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.000129 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 1.29 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 2: Iteration:   500 / 5000 [ 10%] (Warmup)
## Chain 2: Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 2: Iteration:  1500 / 5000 [ 30%] (Warmup)
## Chain 2: Iteration:  2000 / 5000 [ 40%] (Warmup)
## Chain 2: Iteration:  2500 / 5000 [ 50%] (Warmup)
## Chain 2: Iteration:  2501 / 5000 [ 50%] (Sampling)
## Chain 2: Iteration:  3000 / 5000 [ 60%] (Sampling)
## Chain 2: Iteration:  3500 / 5000 [ 70%] (Sampling)
## Chain 2: Iteration:  4000 / 5000 [ 80%] (Sampling)
## Chain 2: Iteration:  4500 / 5000 [ 90%] (Sampling)
## Chain 2: Iteration:  5000 / 5000 [100%] (Sampling)
## Chain 2:

```

```

## Chain 2: Elapsed Time: 9.264 seconds (Warm-up)
## Chain 2:          9.667 seconds (Sampling)
## Chain 2:          18.931 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.000131 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 1.31 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 3: Iteration:   500 / 5000 [ 10%] (Warmup)
## Chain 3: Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 3: Iteration:  1500 / 5000 [ 30%] (Warmup)
## Chain 3: Iteration:  2000 / 5000 [ 40%] (Warmup)
## Chain 3: Iteration:  2500 / 5000 [ 50%] (Warmup)
## Chain 3: Iteration:  2501 / 5000 [ 50%] (Sampling)
## Chain 3: Iteration:  3000 / 5000 [ 60%] (Sampling)
## Chain 3: Iteration:  3500 / 5000 [ 70%] (Sampling)
## Chain 3: Iteration:  4000 / 5000 [ 80%] (Sampling)
## Chain 3: Iteration:  4500 / 5000 [ 90%] (Sampling)
## Chain 3: Iteration:  5000 / 5000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 9.6 seconds (Warm-up)
## Chain 3:          9.815 seconds (Sampling)
## Chain 3:          19.415 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.000123 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 1.23 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 4: Iteration:   500 / 5000 [ 10%] (Warmup)
## Chain 4: Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 4: Iteration:  1500 / 5000 [ 30%] (Warmup)
## Chain 4: Iteration:  2000 / 5000 [ 40%] (Warmup)
## Chain 4: Iteration:  2500 / 5000 [ 50%] (Warmup)
## Chain 4: Iteration:  2501 / 5000 [ 50%] (Sampling)
## Chain 4: Iteration:  3000 / 5000 [ 60%] (Sampling)
## Chain 4: Iteration:  3500 / 5000 [ 70%] (Sampling)
## Chain 4: Iteration:  4000 / 5000 [ 80%] (Sampling)
## Chain 4: Iteration:  4500 / 5000 [ 90%] (Sampling)
## Chain 4: Iteration:  5000 / 5000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 9.902 seconds (Warm-up)
## Chain 4:          10.894 seconds (Sampling)
## Chain 4:          20.796 seconds (Total)
## Chain 4:

```

Model Results

```
print(mod_fit, pars = c("beta", "cutpoints"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=5000; warmup=2500; thin=1;
## post-warmup draws per chain=2500, total post-warmup draws=10000.
##
##               mean se_mean   sd   2.5%   25%   50%   75%  97.5% n_eff Rhat
## beta[1]         3.42    0.00 0.16   3.12   3.31   3.42   3.53   3.74  2830    1
## beta[2]        -0.03    0.00 0.09  -0.21  -0.10  -0.03   0.03   0.15 13475    1
## beta[3]         0.01    0.00 0.12  -0.22  -0.07   0.01   0.09   0.24  8447    1
## beta[4]         0.24    0.00 0.09   0.06   0.18   0.24   0.30   0.42 13645    1
## beta[5]        -0.04    0.00 0.09  -0.22  -0.10  -0.04   0.02   0.13 14575    1
## beta[6]        -0.55    0.00 0.09  -0.72  -0.61  -0.55  -0.49  -0.37 10408    1
## beta[7]         0.12    0.00 0.09  -0.05   0.06   0.12   0.18   0.30 12059    1
## beta[8]         0.06    0.00 0.12  -0.18  -0.02   0.06   0.14   0.30  8302    1
## beta[9]         2.02    0.00 0.13   1.76   1.93   2.02   2.11   2.28  4223    1
## beta[10]        1.96    0.00 0.13   1.70   1.87   1.96   2.04   2.22  5000    1
## beta[11]       13.43    0.01 0.54  12.40  13.07  13.42  13.79  14.51  2365    1
## beta[12]         0.09    0.00 0.11  -0.12   0.02   0.09   0.16   0.29  9076    1
## beta[13]         0.03    0.00 0.11  -0.18  -0.05   0.03   0.10   0.24  9082    1
## beta[14]         0.03    0.00 0.09  -0.14  -0.03   0.03   0.09   0.20 12424    1
## beta[15]        -0.11    0.00 0.18  -0.46  -0.23  -0.11   0.01   0.24 14341    1
## beta[16]        -0.07    0.00 0.17  -0.40  -0.19  -0.07   0.05   0.27 12731    1
## beta[17]         0.03    0.00 0.21  -0.39  -0.11   0.03   0.18   0.45  9153    1
## beta[18]         0.03    0.00 0.25  -0.45  -0.13   0.03   0.20   0.52  9163    1
## beta[19]        -0.06    0.00 0.18  -0.41  -0.18  -0.06   0.06   0.29 13546    1
## beta[20]        -0.28    0.00 0.18  -0.63  -0.40  -0.28  -0.16   0.07 12109    1
## cutpoints[1] -11.90    0.01 0.55 -13.01 -12.26 -11.89 -11.51 -10.83  2645    1
## cutpoints[2]  -0.31    0.00 0.28  -0.85  -0.50  -0.31  -0.13   0.21 11399    1
## cutpoints[3]  11.06    0.01 0.53  10.06  10.70  11.05  11.41  12.12  3262    1
##
## Samples were drawn using NUTS(diag_e) at Sun Dec 14 15:04:43 2025.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

- Initial output from the STAN model

Model Summaries

```
beta_summ <- summary(mod_fit, pars = "beta")$summary
beta_summ <- as.data.frame(beta_summ) %>% rownames_to_column(var = "stan_name")
beta_summ$parameter <- colnames(X)

beta_table <- beta_summ %>%
  dplyr::select(, -`50%`, -`75%`) %>%
  dplyr::select(parameter, everything())
beta_table %>% head()

##      parameter stan_name      mean      se_mean      sd      2.5%
## 1 battery_power  beta[1]  3.41939935  0.0029960304 0.15939037  3.11560573
## 2  clock_speed  beta[2] -0.03243151  0.0008004030 0.09291069 -0.21393494
## 3           fc   beta[3]  0.01159393  0.0012956134 0.11907363 -0.22284253
```

```
## 4    int_memory    beta[4]  0.24175522 0.0007779706 0.09087688 0.05891123
## 5      m_dep      beta[5] -0.04041511 0.0007282533 0.08791997 -0.21541160
## 6    mobile_wt    beta[6] -0.54879395 0.0008634412 0.08808641 -0.72091471
##      25%      97.5%    n_eff      Rhat
## 1  3.30869424  3.7431743  2830.295  1.0017094
## 2 -0.09522202  0.1506866 13474.540  0.9997668
## 3 -0.06851943  0.2415292  8446.568  1.0003822
## 4  0.18044892  0.4175583 13645.218  1.0001833
## 5 -0.09863900  0.1299572 14575.052  1.0001669
## 6 -0.60892119 -0.3739187 10407.641  0.9999245
```

- Model summary table showing us outputs from our model coefficients

```
cp_summ <- summary(mod_fit, pars = "cutpoints")$summary
cp_summ <- as.data.frame(cp_summ) %>% rownames_to_column(var = "stan_name")

cp_summ$parameter <- c("c1", "c2", "c3")

cutpoint_table <- cp_summ %>%
  dplyr::select(, -`50%`, -`75%`) %>%
  dplyr::select(parameter, everything())
cutpoint_table
```

```
##   parameter    stan_name      mean    se_mean      sd      2.5%
## 1      c1 cutpoints[1] -11.8957957 0.010759083 0.5533383 -13.0137768
## 2      c2 cutpoints[2]  -0.3121228 0.002586916 0.2761939  -0.8542437
## 3      c3 cutpoints[3]  11.0579706 0.009231487 0.5272251  10.0593738
##      25%      97.5%    n_eff      Rhat
## 1 -12.2609280 -10.8324497  2645.032  1.001436
## 2  -0.5006295   0.2136466 11398.921  1.000039
## 3  10.6952005  12.1239216  3261.736  1.001039
```

- Model summary table showing us outputs from our model cutpoints

```
t <- bind_rows(beta_table, cutpoint_table)
t
```

```
##      parameter    stan_name      mean    se_mean      sd      2.5%
## 1  battery_power    beta[1]   3.41939935 0.0029960304 0.15939037   3.11560573
## 2   clock_speed    beta[2] -0.03243151 0.0008004030 0.09291069  -0.21393494
## 3          fc      beta[3]   0.01159393 0.0012956134 0.11907363  -0.22284253
## 4   int_memory    beta[4]   0.24175522 0.0007779706 0.09087688   0.05891123
## 5      m_dep      beta[5] -0.04041511 0.0007282533 0.08791997  -0.21541160
## 6    mobile_wt    beta[6] -0.54879395 0.0008634412 0.08808641  -0.72091471
## 7      n_cores    beta[7]   0.12302977 0.0008158201 0.08958840  -0.05321055
## 8          pc      beta[8]   0.06280944 0.0013365281 0.12178095  -0.17534896
## 9   px_height    beta[9]   2.01788204 0.0020333124 0.13213552   1.76386174
## 10   px_width    beta[10]   1.95653479 0.0018401388 0.13011880   1.70380163
## 11          ram    beta[11]  13.43345336 0.0110470183 0.53725366  12.40281138
## 12       sc_h     beta[12]   0.08645143 0.0011063509 0.10540226  -0.12093166
## 13       sc_w     beta[13]   0.02727717 0.0011345463 0.10811979  -0.18180432
## 14   talk_time    beta[14]   0.02770810 0.0007810909 0.08706126  -0.14276790
## 15        blue    beta[15] -0.10653924 0.0014814868 0.17741385  -0.46063783
## 16   dual_sim     beta[16] -0.06688324 0.0015366632 0.17338146  -0.39886140
## 17       four_g    beta[17]   0.03103094 0.0022285828 0.21320855  -0.39297701
## 18       three_g    beta[18]   0.03281379 0.0025893949 0.24786011  -0.44986753
## 19  touch_screen    beta[19] -0.06269599 0.0015252495 0.17752079  -0.40677312
```

```
## 20      wifi      beta[20] -0.28196768 0.0016206183 0.17833426 -0.63294440
## 21      c1 cutpoints[1] -11.89579575 0.0107590832 0.55333826 -13.01377684
## 22      c2 cutpoints[2] -0.31212281 0.0025869158 0.27619395 -0.85424374
## 23      c3 cutpoints[3] 11.05797057 0.0092314873 0.52722513 10.05937385
##          25%          97.5%      n_eff      Rhat
## 1      3.30869424      3.74317426 2830.295 1.0017094
## 2     -0.09522202      0.15068659 13474.540 0.9997668
## 3     -0.06851943      0.24152923  8446.568 1.0003822
## 4      0.18044892      0.41755829 13645.218 1.0001833
## 5     -0.09863900      0.12995719 14575.052 1.0001669
## 6     -0.60892119    -0.37391869 10407.641 0.9999245
## 7      0.06337563      0.30039759 12059.097 0.9999983
## 8     -0.01930600      0.29900811  8302.379 1.0000856
## 9      1.92941780      2.28083035  4223.096 1.0006765
## 10     1.86821169      2.21856626 5000.103 1.0005602
## 11    13.06546282    14.50866730 2365.204 1.0014986
## 12     0.01502517      0.29446027  9076.410 0.9997959
## 13    -0.04742870      0.23962681  9081.675 1.0000321
## 14    -0.03148995      0.19669226 12423.577 0.9999813
## 15    -0.22666643      0.23837570 14341.000 0.9999183
## 16    -0.18604319      0.27447726 12730.572 1.0000200
## 17    -0.11449539      0.44889849  9152.752 1.0000090
## 18    -0.13463055      0.51696206  9162.557 1.0000248
## 19    -0.18241934      0.28709577 13546.174 0.9996962
## 20    -0.40174345      0.07132849 12108.995 1.0000226
## 21   -12.26092804   -10.83244967  2645.032 1.0014364
## 22    -0.50062950      0.21364658 11398.921 1.0000393
## 23    10.69520053    12.12392159  3261.736 1.0010394
```

- Combined model summary table showing us outputs from our model coefficients and cutpoints

```
beta_draws <- as.matrix(mod_fit, pars = "beta")
colnames(beta_draws) <- colnames(X)
# beta_draws
```

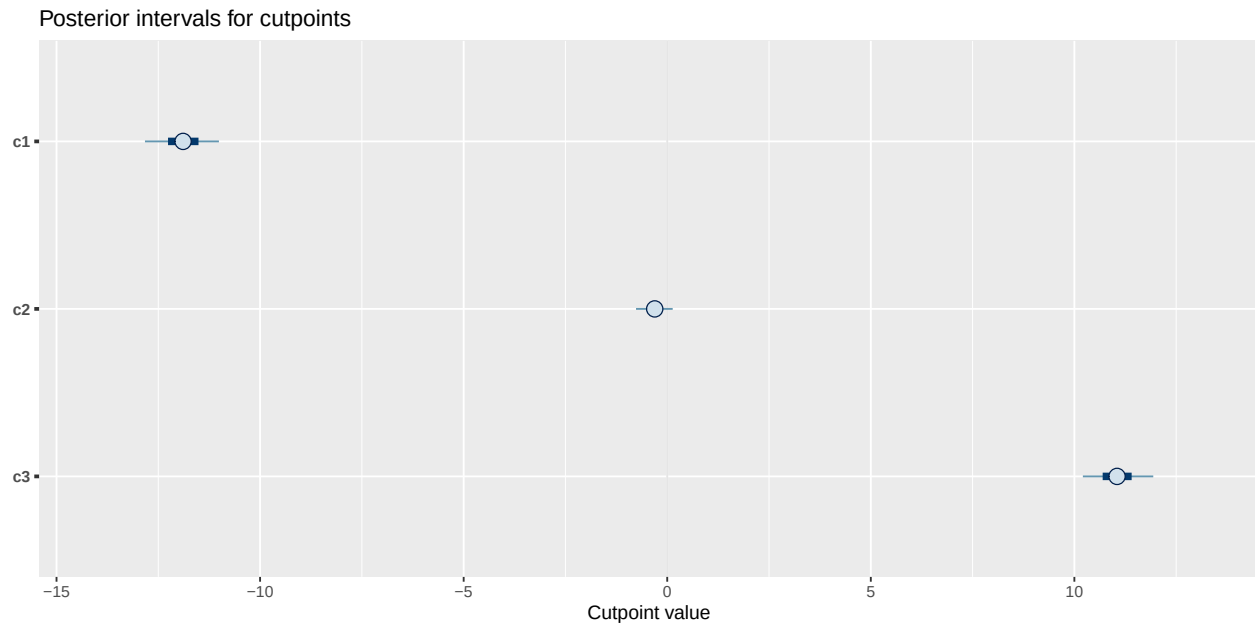
- Changing coefficient names back to our variable names for better plotting

Interval Plots

```
cp_draws <- as.matrix(mod_fit, pars = "cutpoints")
colnames(cp_draws) <- c("c1", "c2", "c3")

cut_int <- mcmc_intervals(cp_draws) +
  ggplot2::labs(
    title = "Posterior intervals for cutpoints",
    x = "Cutpoint value"
  )

cut_int
```



```
ggsave("cut_interval_plots.png", dpi = 600)
```

```
## Saving 10 x 5 in image
```

- Cutpoint interval plot showing good results

```
idata <- mcmc_intervals_data(beta_draws,
                             prob = 0.5,
                             prob_outer = 0.95) %>%
  mutate(includes_zero = (ll <= 0 & hh >= 0))

new_coef_int <- ggplot(idata, aes(y = reorder(parameter, m), x = m)) +
  geom_vline(xintercept = 0, linetype = "dashed", colour = "grey50") +
  # 95% interval
  geom_errorbarh(aes(xmin = ll, xmax = hh,
                    colour = includes_zero),
                height = 0) +
  # 50% interval
  geom_errorbarh(aes(xmin = l, xmax = h,
                    colour = includes_zero),
                height = 0, linewidth = 1.4) +
  geom_point(aes(colour = includes_zero), size = 2) +
  scale_colour_manual(
    values = c(`FALSE` = "black", `TRUE` = "grey70"),
    labels = c(`FALSE` = "95% CI excludes 0",
               `TRUE` = "95% CI includes 0"),
    name = NULL
  ) +
  labs(
    x = "Coefficient",
    y = NULL,
    title = "Posterior intervals for regression coefficients"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```

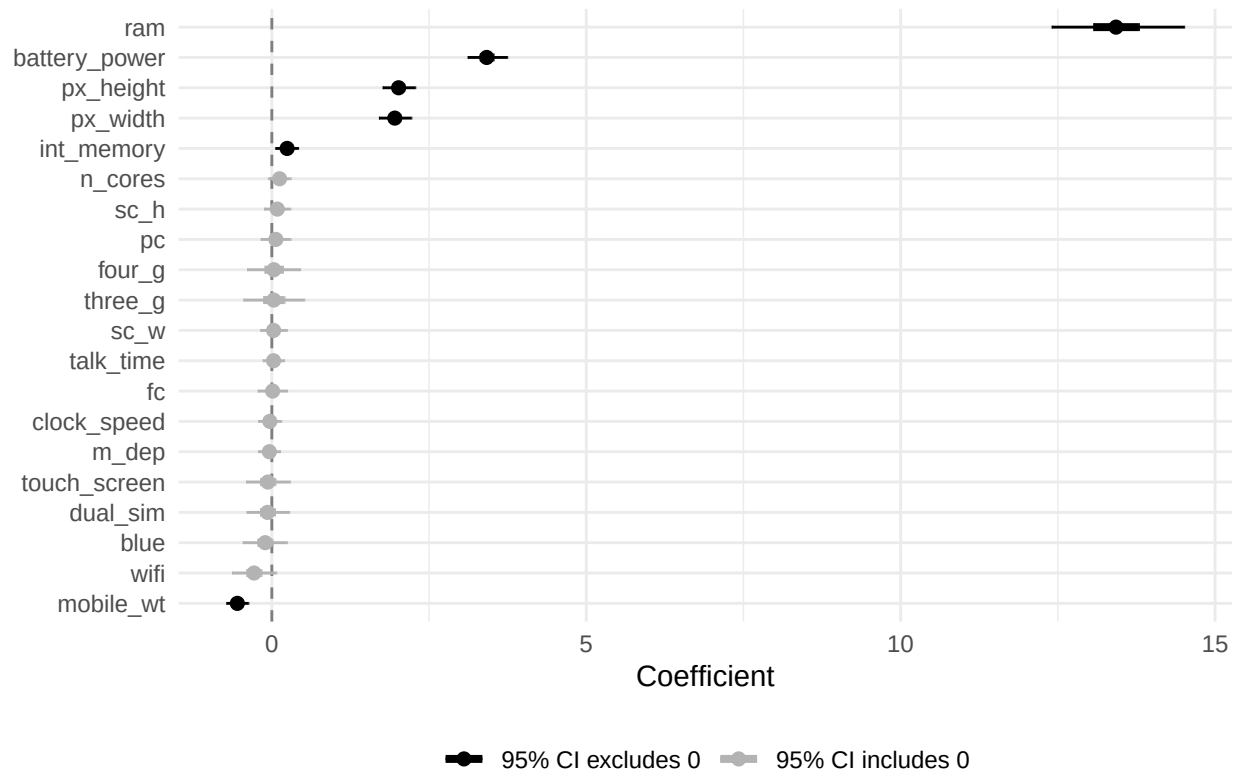
```
## Warning: `geom_errorbarh()` was deprecated in ggplot2 4.0.0.
## i Please use the `orientation` argument of `geom_errorbar()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
new_coef_int
```

```
## `height` was translated to `width`.
```

```
## `height` was translated to `width`.
```

Posterior intervals for regression coefficients



```
ggsave("beta_new_interval.png", dpi=600)
```

```
## Saving 6.5 x 4.5 in image
```

```
## `height` was translated to `width`.
```

```
## `height` was translated to `width`.
```

- Coefficient interval plot showing our significant variables

```
g <- arrangeGrob(new_coef_int, cut_int, ncol = 2) #generates g
```

```
## `height` was translated to `width`.
```

```
## `height` was translated to `width`.
```

```
ggsave(file="new_interval_plots.png", g, dpi=600)
```

```
## Saving 10 x 5 in image
```

Trace Plot

```
all_arr <- as.array(mod_fit, pars = c("beta", "cutpoints"))
```

```

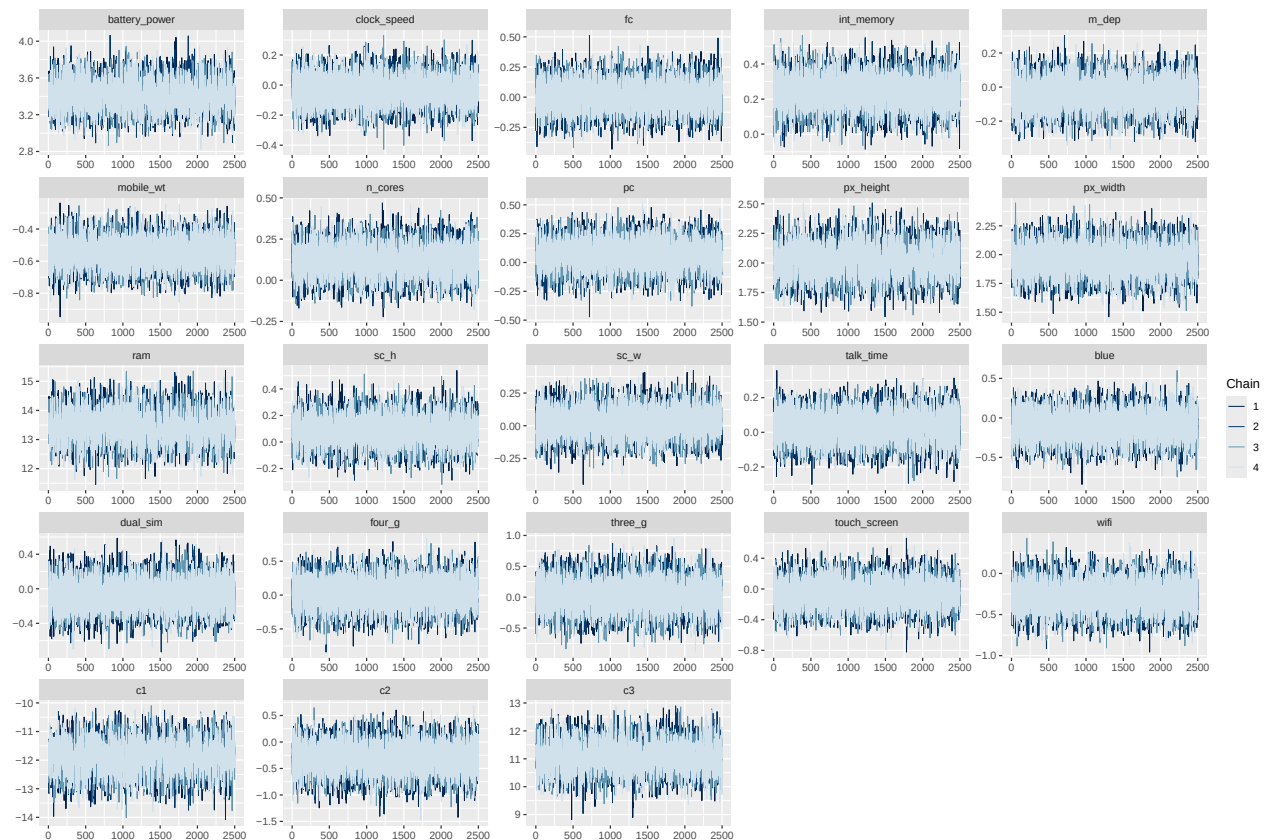
D    <- ncol(X)    # number of predictors
K    <- 4          # number of categories
beta_names <- colnames(X)    # "battery_power", "ram"
cp_names  <- paste0("c", 1:(K - 1))    # "c1", "c2", "c3"

dimnames(all_arr)[[3]] <- c(beta_names, cp_names)

# rstan::traceplot(mod_fit, pars = c("beta", "cutpoints"))

mcmc_trace(
  all_arr,
  pars = c(beta_names, cp_names)
)

```



```
ggsave("trace_plots.png", dpi = 600)
```

Saving 15 x 10 in image

- Trace plots of all coefficients and cutpoints showing good mixing and convergence

Posterior Predictive Draws and Checks

```

color_scheme_set("brightblue")

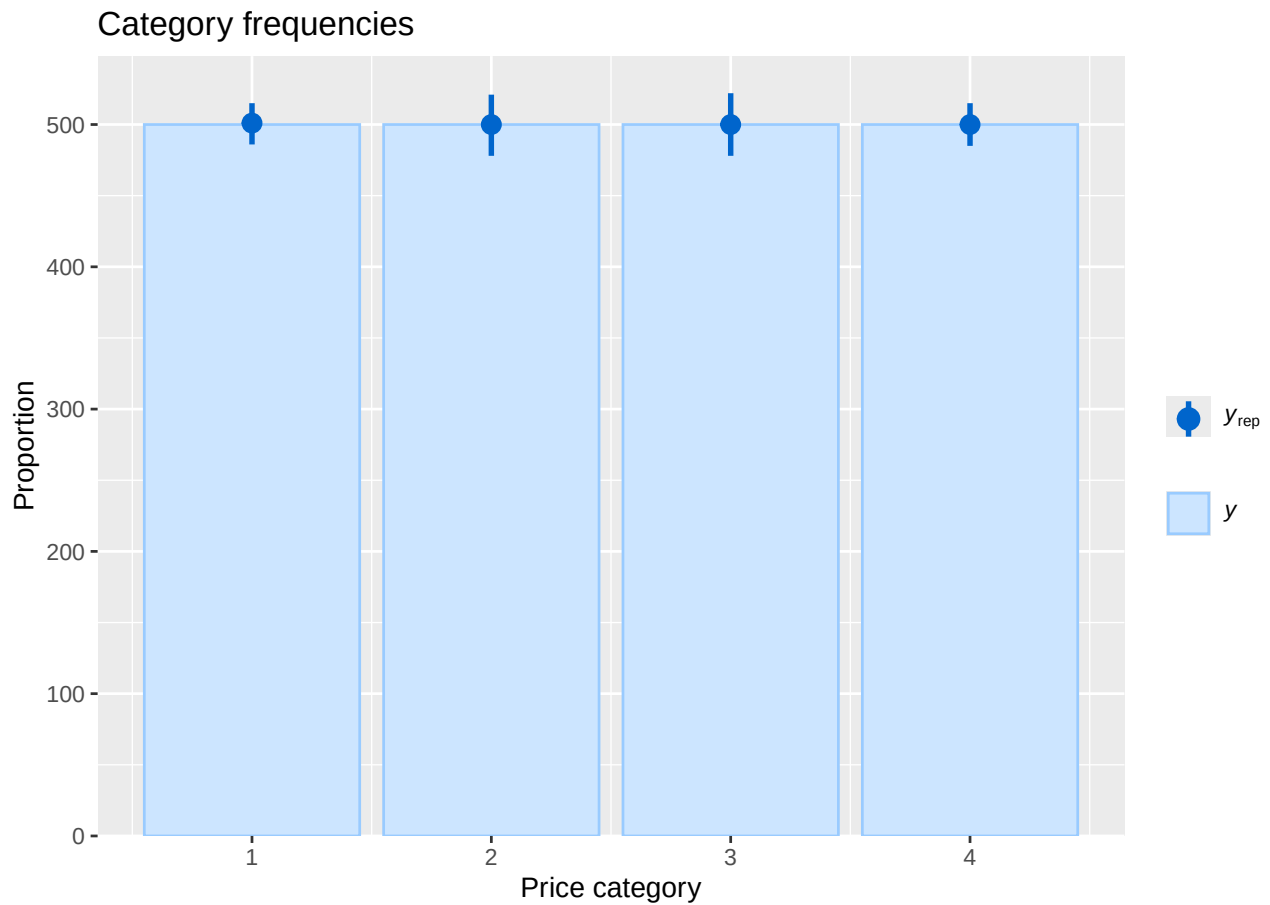
y <- data$y
K <- data$K

```

```
yrep_basic <- rstan::extract(mod_fit, "y_pred")$y_pred
```

```
freq_ppc <- ppc_bars(
  y = y,
  yrep = yrep_basic
) +
  ggplot2::labs(
    title = "Category frequencies",
    x = "Price category", y = "Proportion"
  )
```

```
freq_ppc
```



```
ggsave("ppc_category_freq.png", dpi=600)
```

```
## Saving 7 x 5 in image
```

- Posterior predictive checks on price categories. Shows that the model accurately captures the empirical distribution

```
mean_ppc <- ppc_stat(
  y = y,
  yrep = yrep_basic,
  stat = "mean"
) +
  ggplot2::labs(
```

```

    title = "Mean price category"
  )

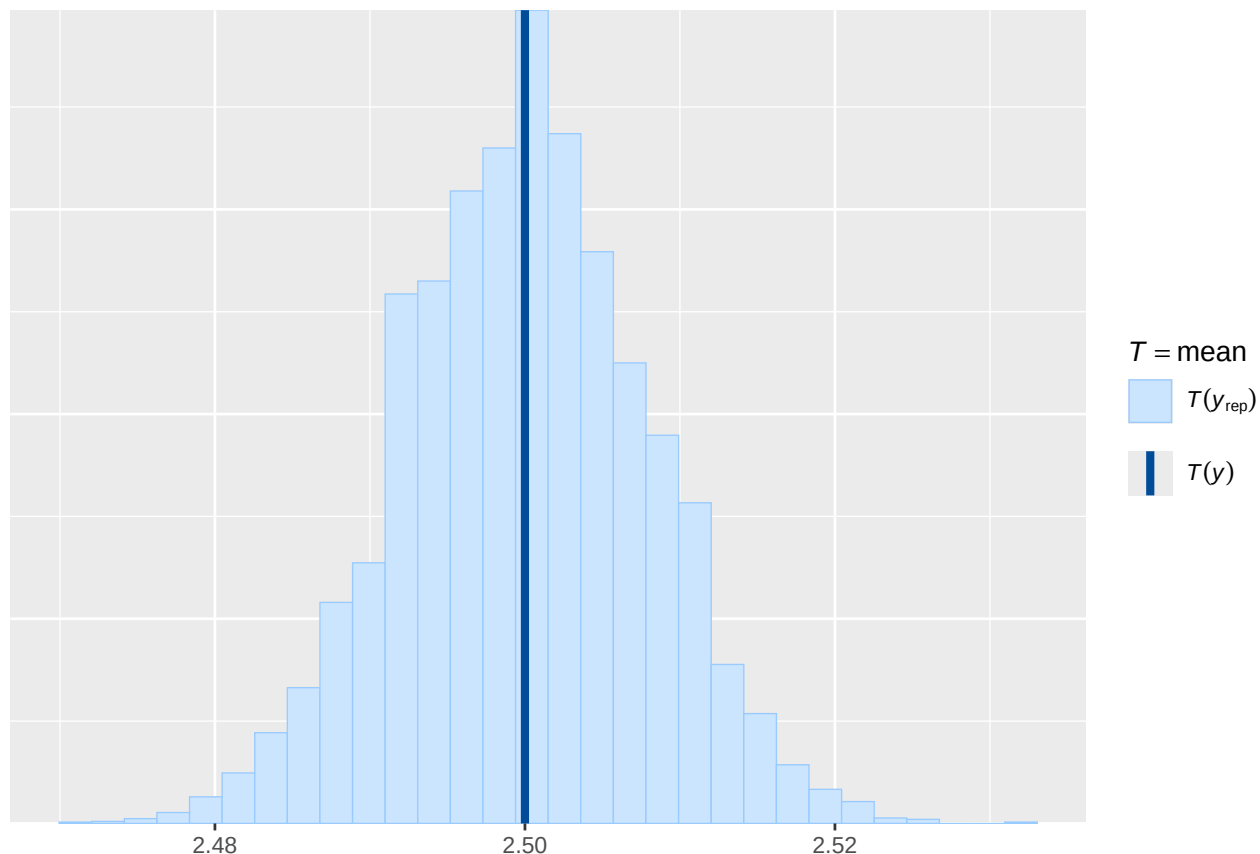
```

Note: in most cases the default test statistic 'mean' is too weak to detect anything of interest.

```
mean_ppc
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

Mean price category



```
ggsave("ppc_mean.png", dpi=600)
```

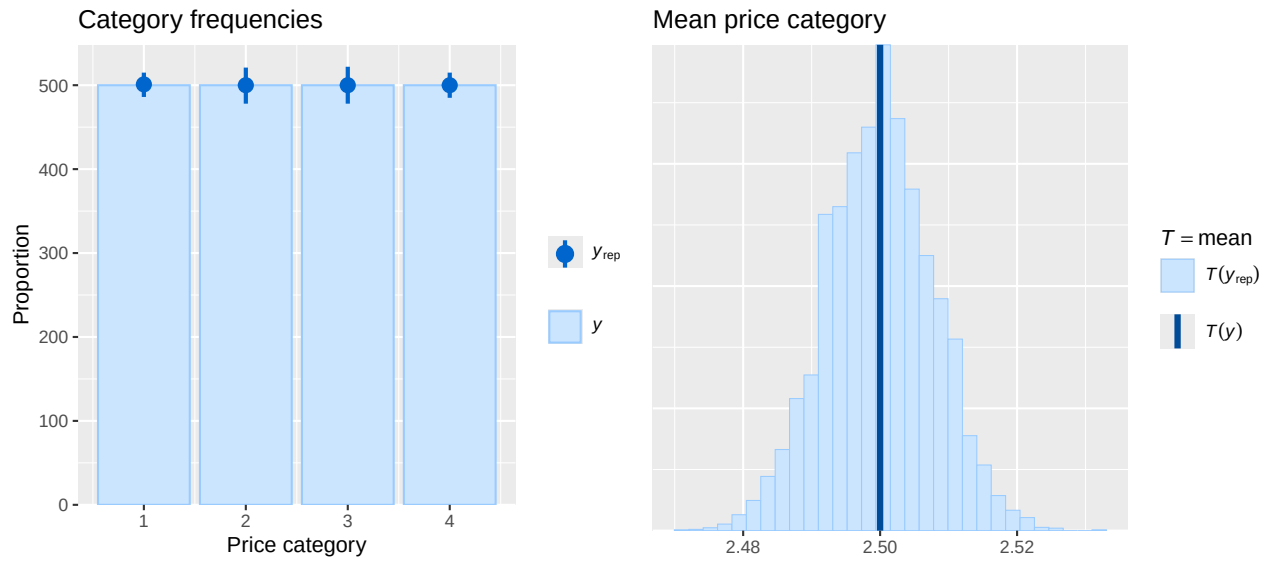
Saving 7 x 5 in image

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

- Posterior predictive check on the mean. We find that the mean of the data is accurately captured in our posterior distribution indicating no over or under estimation

```
grid.arrange(freq_ppc, mean_ppc, ncol=2)
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.



```
g <- arrangeGrob(freq_ppc, mean_ppc, ncol = 2)

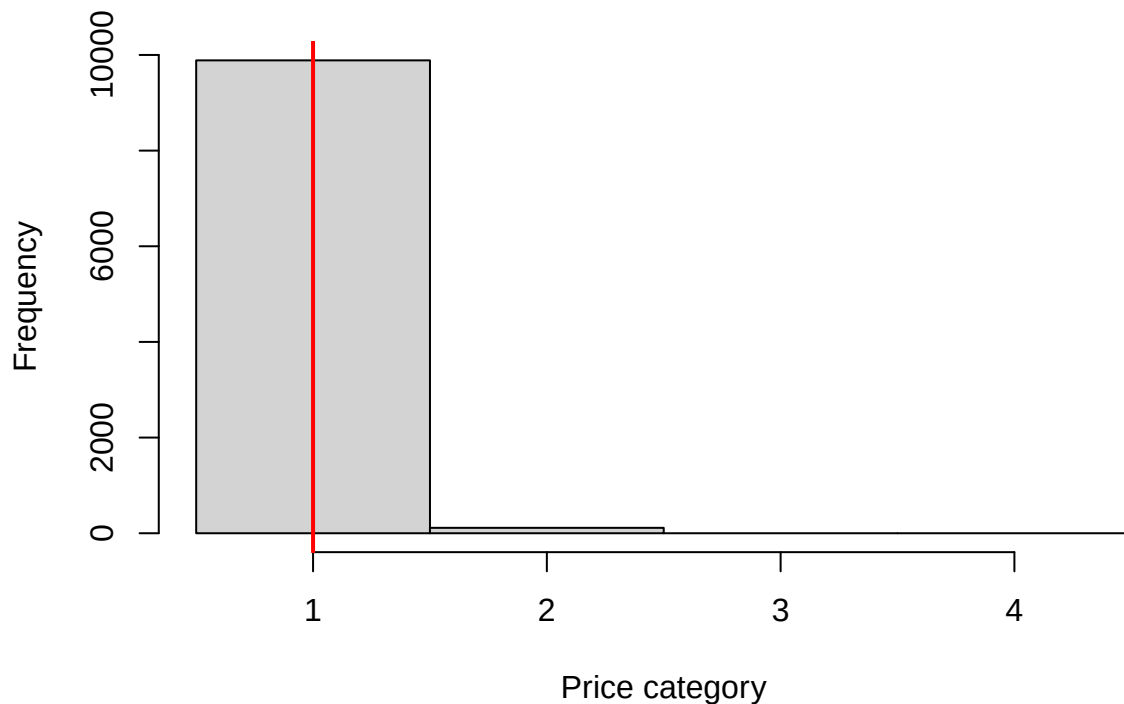
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
ggsave(file="ppc_basics.png", g, dpi=600)

## Saving 9 x 4 in image
i <- c(97, 26, 27, 28, 29, 45, 67, 89, 19)
i <- 57
# draws <- sample(seq_len(dim(yrep_basic)[1]), 500)
draws <- dim(yrep_basic)[1]

# png(filename = "phone_pred.png", width = 800, height = 600, units = "px", res = 100)

hist(
  yrep_basic[, i],
  breaks = seq(0.5, 4.5, by = 1),
  main = paste("Posterior predictive for phone", i),
  xlab = "Price category"
)
abline(v = y[i], col = "red", lwd = 2)
```

Posterior predictive for phone 57



```
# dev.off()
```

- Example plot of a phone prediction. Red is the true category. Shows that the model accurately predicts training set phones

```
S <- dim(yrep_basic)[1]
N <- dim(yrep_basic)[2]

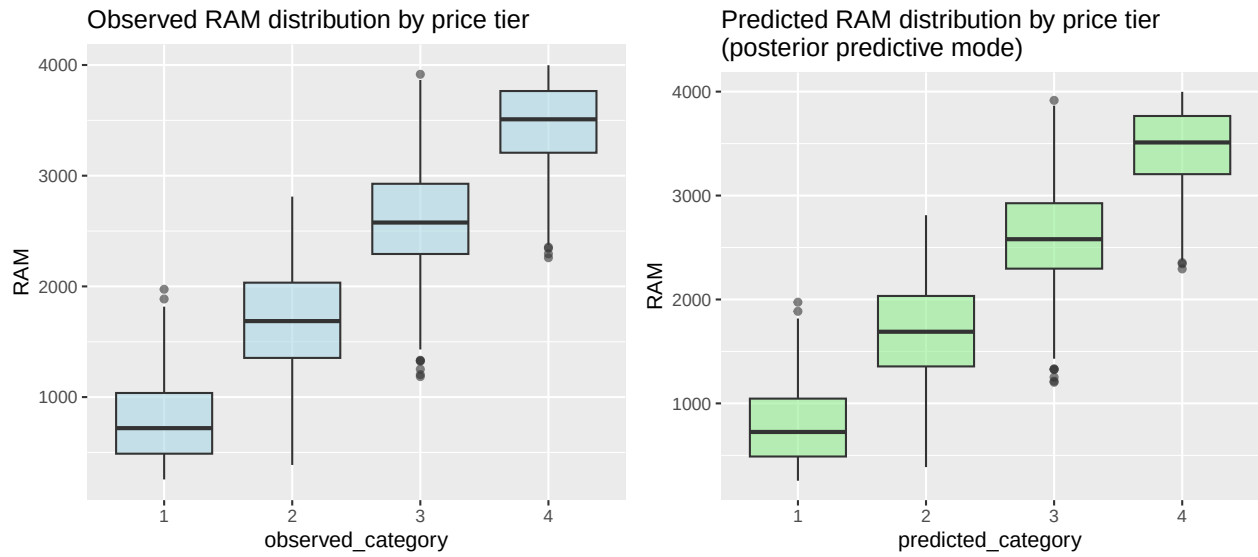
y_pred_mode <- apply(yrep_basic, 2, function(x) {
  as.numeric(names(sort(table(x), decreasing = TRUE)[1]))
})

df_check <- data.frame(
  RAM = train$ram,
  observed_category = factor(y, levels = 1:K),
  predicted_category = factor(y_pred_mode, levels = 1:K)
)

orig_ram_price <- ggplot(df_check, aes(x = observed_category, y = RAM)) +
  geom_boxplot(fill = "lightblue", alpha = .6) +
  labs(title = "Observed RAM distribution by price tier")

ppc_ram_price <- ggplot(df_check, aes(x = predicted_category, y = RAM)) +
  geom_boxplot(fill = "lightgreen", alpha = .6) +
  labs(title = "Predicted RAM distribution by price tier\n(posterior predictive mode)")

g <- arrangeGrob(orig_ram_price, ppc_ram_price, ncol = 2)
grid.arrange(orig_ram_price, ppc_ram_price, ncol = 2)
```



```
ggsave(file="ram_price_ppc.png", g, dpi=600)
```

Saving 9 x 4 in image

- We compared the distribution of RAM across observed categories to the distribution under the posterior predictive price assignments
 - We find that the model reproduces the monotonic trend between RAM and price tier, implying that the estimated coefficients faithfully capture the relationship between hardware specifications and price structure and that the model is capable of generating realistic phone price distributions